

Figure 3: The interrupt process

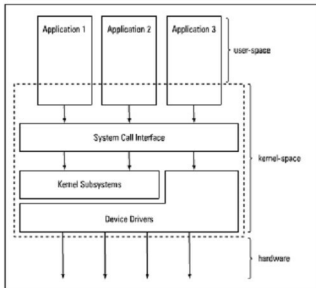


Figure 4: The generalized architecture

the second one is for serialising the **message writes**. The size of the message is found when the message queue is generated.

Out of all these, shared memory is supposed to be the fastest IPC mechanism. As we discussed, this IPC allows processes to share the physical memory. MMS (memory management system) does the creation of the shared physical memory regions. The system call `sys_shmat()` links the shared pages to the user processes virtual memory space.

## Data structures (for IPC mechanisms)

Now, we can talk about the data structures to implement these IPC mechanisms. The implementation of signals is done using the signal field in the **task\_struct** structure. And every signal is conveyed using a bit in the field. So the total number of signals is limited to the number of bits in a word.

The **wait\_queue** structure, which has a pointer to the associated **task\_struct**, is the one linked to wait queues. **Inode** is the filesystem for pipes. It records the extra pipe-specific information in the **pipe\_inode\_info**. And it has a wait queue (for the processes), the quantity of data, and the number of processes that are reading (/writing) from (/to) the pipe.

For **file locks** we have **file\_lock** structure. This has a pointer to a **task\_struct** for the owning process, a file descriptor of the locked file, and a wait queue for processes and the area where it is locked. For every open file, the **file\_lock** structures will link to a list. And socket data structure is used for representing the UNIX domain sockets.

When it comes to the system V IPC objects, we can see that these are created in the kernel. The associated access permissions are mentioned in the **ipc\_perm** structure. Semaphores are represented with the **sem** structure (which also has the value and the PID of the process that used semaphore). The arrays are done using the **semid\_ds** structure (that carries access permissions and the time of the last operation, the pointer to the first semaphore in the list, etc). The **Sem\_undo** (structure) is employed to get an array of semaphore operations performed by a process (used during the killing of the process).

Message queues are represented by the **msgqid\_ds** structure, which also has the control and management information. It can store the following:

- access permissions
- the current number of bytes in the queue
- the number of messages
- the size of the queue
- the process number of the last message sent
- the process number of the last message received
- the time of the last change and message sent and received
- link fields to implement the message queue

And a message is recorded in the kernel with a **msg** structure. This has information about the link field (for the link list), the message type, address of data (message data) and the length. The **Shmid\_ds** structure is used to represent the shared memory implementation. It has access control permissions, PIDs of the creator, number of processes to which the shared physical memory region (and number of pages that make this zone) is linked, detach and change times, etc.

We can now summarise the overall architecture using Figure 4. (You can see that this is a very generalised view to just put the idea in a nutshell; for the exact structure, please refer to the earlier columns.)

## Newbie zone

I'm going to discuss a few things that I have been telling novice programmers. First of all, let's look at the differences between Linux and the UNIX kernel. Since Linux is based on the UNIX architecture, there are many similarities. But Linux is not just a copy of UNIX.

The most interesting feature of Linux is that it allows you to load modules dynamically. We have already learned how to write a module and load that to the kernel. This feature is remarkable especially when you find that the kernel is monolithic in nature, but can still support this one feature. Another characteristic that needs special emphasis is the treatment of threads in Linux. It views threads and normal processes with the same eye. It has symmetrical multi-processor (SMP) support (though this is available in